EX16b - Mask Practice - AND, OR, NOT, any and all

Beginner Step-by-Step Guide

This guide follows the real file `ex16b_masks_filtering_practice.py`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Practice filtering a [time, altitude, velocity, temperature] dataset using simple and compound masks.

Problem variables / elements

Name	Meaning
sensor_data	9x4 matrix
mask	condition per row

Equation or logic on paper

```
mask = condition
values = vector[mask]
rows = matrix[mask]
```

Manual check

altitudes>=100 is True at indices [3,4,5,6], so there are four high-altitude samples.

2. What we already know and what is new

Already seen:

- masks
- A[mask]
- np.where
- np.diff

New in this exercise:

- extract columns
- x[mask] vs A[mask]
- |
- ~
- np.min
- np.abs
- np.any
- np.all

3. New Python concepts

1. Values vs rows

Syntax: `altitudes[mask] vs sensor_data[mask]`

On a vector we get values; on the full matrix we get rows.

2. OR

Syntax: `mask_a | mask_b`

True if at least one condition is True at that position.

3. NOT

Syntax: `~mask`

Flips each Boolean. ~(velocities>0) also includes zero velocity.

4. any

Syntax: `np.any(mask)`

True if at least one value is True.

5. all

Syntax: `np.all(mask)`

True only if every value is True.

4. Code step by step

Step 1. Extract columns

```
times = sensor_data[:, 0]
altitudes = sensor_data[:, 1]
velocities = sensor_data[:, 2]
temperatures = sensor_data[:, 3]
```

Naming columns makes later expressions easier to read.

Step 2. Simple masks

```
altitude_equal_above_100 = altitudes >= 100
velocity_positive = velocities > 0
```

Each result is a vector of nine Booleans.

Step 3. Values and rows

```
high_altitude_values = altitudes[altitude_equal_above_100]
high_altitude_states = sensor_data[altitude_equal_above_100]
```

The same mask selects different structures depending on where it is applied.

Step 4. AND and OR

```
(altitudes >= 100) & (velocities > 0)
(altitudes >= 100) | (temperatures < 12)
```

& requires both conditions; | requires at least one.

Step 5. NOT

```
not_ascending = ~(velocities > 0)
```

This means not positive, i.e. <=0; it is not exactly the same as velocities<0.

Step 6. Derived variable

```
speed = np.abs(velocities)
processed_data = np.column_stack((sensor_data, speed))
```

np.abs removes sign to create speed; column_stack adds it to the dataset.

Step 7. Global questions

```
np.any(altitudes > 130)
np.all(temperatures > 0)
```

These functions reduce a complete mask to one Boolean answer.

5. Execution flow

```
sensor matrix
 -> named columns
 -> simple masks
 -> values/rows
 -> AND/OR/NOT
 -> statistics / derived column
 -> any/all
```

6. Expected result / check

The file finds 4 samples with altitude>=100, 5 with velocity>0, 4 with velocity<0 and returns YES for any altitude>130 and all temperature>0.

7. Common beginner mistakes

- Interpreting ~(v>0) as strictly v<0.
- Using & or | without parentheses.
- Confusing values with rows.
- Confusing np.any with np.all.

8. If you remember only five things

- The same mask can filter a vector or a matrix.
- &=AND, |=OR, ~=NOT.
- np.abs converts signed velocity to speed.
- np.any asks whether any is True.
- np.all asks whether all are True.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.